Software Architecture Document for the Electronics Store Inventory Management System (SEIMS)

Karthik Peddibhotla
Department of Computer Science & Engineering
PES University
SRN: PES2UG24CS348
karthikpeddibhotla5@gmail.com

Paridhi Bajpai
Department of Computer Science & Engineering
PES University
SRN: PES2UG24CS337
paridhi.bajaj03@gmail.com

Niharika S Murthy
Department of Computer Science & Engineering
PES University
SRN: PES2UG24CS316
niharika.sm0@gmail.com

P. Dhanush
Department of Computer Science & Engineering
PES University
SRN: PES2UG24CS333
penchalpaadudhanush@gmail.com

***Abstract*—This Software Architecture Document (SAD) describes the architecture of the Electronics Store Inventory Management System (SEIMS). It presents the system organization, technology stack, major software components, database structure, request workflows, security considerations, API design, testing organization, and deployment approach. The system uses a Flask-based web application with an SQLite relational database and server-rendered HTML views. The architecture separates presentation, application processing, and data persistence to support maintainability, reliability, and testability.**

***Index Terms*—Software Architecture Document, SEIMS, three-layer architecture, Flask, SQLite, component design, security, inventory management.**

I. Introduction

A. Purpose

This Software Architecture Document describes the high-level design of the Electronics Store Inventory Management System. It explains the system's major structures, architectural decisions, component responsibilities, database organization, workflows, interfaces, and security mechanisms. It complements the Software Requirements Specification (SRS) and provides a technical reference for development, testing, and maintenance.

B. Scope

The architecture covers Flask request handling, server-side template rendering, SQLite database access, authentication and session handling, inventory operations, stock classification, reporting, CSV export, automated testing, and the intended deployment model.

C. Definitions and Acronyms

D. Document Organization

The remainder of this document describes the architectural overview, component design, database architecture, detailed workflows, API design, security architecture, testing, deployment, quality attributes, risks, and references.

TABLE I
Architectural Acronyms

Term	Meaning
SAD	Software Architecture Document
SRS	Software Requirements Specification
MVC	Model-View-Controller pattern
WSGI	Web Server Gateway Interface
SKU	Stock Keeping Unit
CRUD	Create, Read, Update, Delete
API	Application Programming Interface

II. Architectural Overview

A. System Overview

SEIMS follows a three-layer web architecture. The presentation layer renders browser-facing pages using HTML, CSS, and Jinja2 templates. The application layer uses Python and Flask to handle routes, sessions, validation, and business rules. The data layer uses SQLite to persist user and component records.

The browser communicates with the Flask application through HTTP requests. Flask processes requests, interacts with the database, and returns rendered HTML pages or downloadable CSV responses.

B. Architectural Pattern

The system uses a layered architectural pattern with responsibilities separated into presentation, application, and data layers. Flask routes coordinate request processing and application logic, HTML templates provide the user interface, and SQLite manages persistent records.

The design also follows selected Model-View-Controller principles:

- **Model:** Inventory and user data stored in SQLite.
- **View:** HTML templates rendered using Jinja2.
- **Controller:** Flask routes and request handlers.

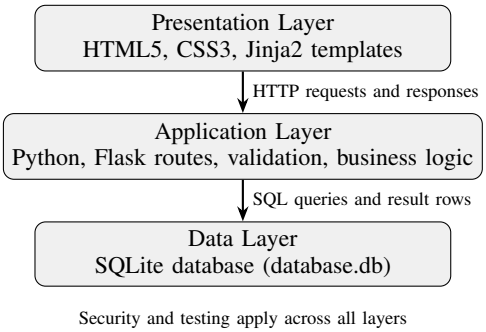


Fig. 1. Three-Layer Architecture of SEIMS

TABLE II
TECHNOLOGY STACK MAPPING

Area	Technology	Responsibility
Presentation	HTML, CSS, Jinja2	User interface rendering
Application	Python, Flask	Routing and business logic
Database	SQLite 3	Relational data persistence
Security	Werkzeug	Password hashing utilities
Testing	Pytest	Automated verification
Version Control	Git, GitHub	Source control and collaboration
CI	GitHub Actions	Automated test workflow

C. Architectural Goals

The architecture is guided by the following goals:

- **Simplicity and maintainability:** Separate templates, styles, backend routing, and database operations.
- **Data integrity:** Apply database constraints for unique SKUs and valid stock and price values.
- **Security baseline:** Use password hashing and authenticated-session checks for protected routes.
- **Testability:** Verify application behavior through automated tests.
- **Usability:** Provide clear inventory views, dashboard summaries, filters, and reports.

D. Technology Stack

III. COMPONENT DESIGN

A. Component Breakdown

The application is organized around the following functional components:

- **Authentication Component:** Verifies credentials and manages the user session.
- **Dashboard Component:** Retrieves and displays inventory summary counts.
- **Inventory Display Component:** Presents component records and supports search and filtering.

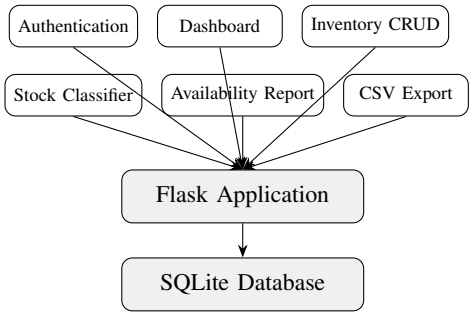


Fig. 2. Logical Component Architecture

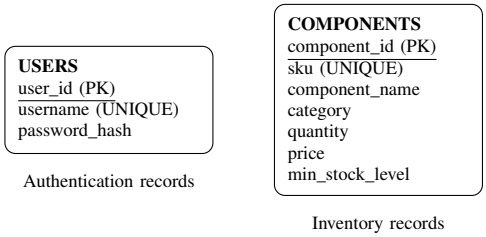


Fig. 3. Logical Database Entities

- **Inventory Management Component:** Creates, reads, updates, and deletes component records.
- **Stock Classifier:** Determines component status using quantity and minimum stock level.
- **Availability Reporter:** Groups inventory records by stock status.
- **CSV Exporter:** Produces downloadable CSV inventory data.
- **Database Handler:** Manages SQLite connections and database queries.

B. Component Interaction

Fig. 2 shows the logical relationships between the functional components, Flask application, and SQLite database. These components represent logical responsibilities and may be implemented within the main Flask application module.

C. Component Responsibilities

The authentication component verifies user credentials before allowing access to protected functions. Inventory management handles component creation, modification, deletion, and retrieval. The stock classifier evaluates component quantities against configured minimum stock levels. The reporting component groups components by status, while the CSV exporter prepares inventory information for download.

IV. DATABASE ARCHITECTURE

A. Data Model

The database stores user credentials as password hashes and component inventory details. The Users table stores account information. The Components table stores identifying information, category, quantity, price, and minimum stock level.

TABLE III
COMPONENTS TABLE SCHEMA

Column	Type	Constraint
component_id	INTEGER	Primary key, auto-increment
sku	TEXT	Unique identifier
component_name	TEXT	Required component name
category	TEXT	Required category
quantity	INTEGER	Non-negative value
price	REAL	Non-negative value
min_stock_level	INTEGER	Default 10, non-negative

TABLE IV
USERS TABLE SCHEMA

Column	Type	Constraint
user_id	INTEGER	Primary key, auto-increment
username	TEXT	Required and unique
password_hash	TEXT	Required password hash

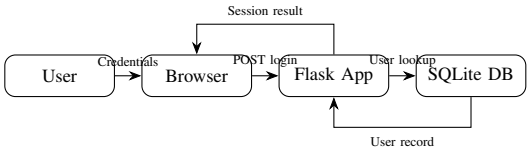


Fig. 4. Authentication Request Workflow

B. Components Table Schema

C. Users Table Schema

D. Data Integrity

The database design uses primary keys to identify records and uniqueness constraints to prevent duplicate usernames and SKUs. Quantity, price, and minimum stock level are expected to be non-negative. Application-level validation and database constraints work together to maintain data integrity.

V. DETAILED DESIGN AND WORKFLOWS

A. Authentication Workflow

The authentication workflow begins when a user submits login credentials through the browser. Flask receives the request, retrieves the matching user record from SQLite, verifies the submitted password against the stored password hash, and establishes an authenticated session when verification succeeds. If credentials are invalid, the application returns an authentication failure response.

B. Inventory Update Workflow

For an inventory update, the authenticated user submits revised component information. Flask validates the input and performs a parameterized database update. The application then returns an updated page or an appropriate validation message. Database constraints provide additional protection for values such as SKU uniqueness and non-negative quantity and price.

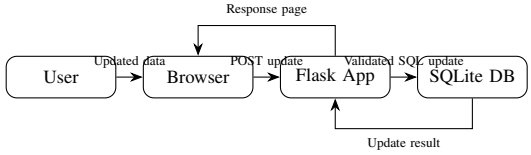


Fig. 5. Inventory Update Workflow

TABLE V
LOGICAL APPLICATION INTERFACE DESIGN

Operation	Method	Purpose
Login	GET/POST	Display login and process credentials
Logout	GET/POST	End the authenticated session
Dashboard	GET	Display inventory summaries
Inventory	GET	Display and filter components
Add component	GET/POST	Display form and create a record
Edit component	GET/POST	Display form and update a record
Delete component	POST	Delete a selected record
Availability report	GET	Display stock classifications
CSV export	GET	Download inventory data

VI. API AND INTERFACE DESIGN

A. Browser Interface

The browser interface provides pages for authentication, dashboard summaries, inventory display, component creation, component editing, and availability reporting. Templates are rendered by Flask using Jinja2.

B. Application Interfaces

The Flask application exposes browser-facing routes for user authentication, inventory operations, dashboard information, reports, and CSV export. The exact route names and HTTP methods are determined by the implemented application.

The table describes logical application operations. Actual route paths and methods should be checked against the current Flask implementation before submission.

C. Communication Interface

The browser communicates with the Flask application through HTTP requests and responses. The local development environment uses the configured Flask development port. A production deployment should use HTTPS.

VII. ERROR HANDLING

The application should handle errors in a controlled manner to prevent invalid operations and provide understandable feedback.

- **Authentication errors:** Invalid credentials should result in an authentication failure message.
- **Validation errors:** Missing or invalid component values should be rejected with appropriate feedback.

TABLE VI
AUTOMATED TEST SUITE STRUCTURE

Test File	Focus Area
test_app.py	Routes, sessions, and CSV export
test_frontend.py	Layouts and template rendering
test_inventory_backend.py	Database CRUD and stock-classifier logic

- **Database errors:** Database operation failures should be handled without exposing internal database details to users.
- **Missing records:** Requests for unavailable component records should return an appropriate not-found response or message.
- **Access errors:** Unauthenticated users should be redirected to the login page or denied access to protected operations.

VIII. SECURITY ARCHITECTURE

The security architecture addresses confidentiality, integrity, and controlled access to inventory functions.

- **Password protection:** Passwords are stored using Werkzeug password-hashing utilities rather than plaintext passwords.
- **Session guarding:** Protected endpoints use the application's authentication and session checks.
- **SQL injection defence:** Database operations should use parameterized SQLite queries rather than directly concatenating untrusted input into SQL statements.
- **Input constraints:** Application validation and database constraints reject invalid values, including negative quantities and duplicate SKUs.
- **Secret management:** Production deployment should use a securely generated secret key supplied through environment configuration.
- **Transport security:** HTTPS should be configured for production deployment to protect credentials and session traffic in transit.

The production secret-key and HTTPS requirements describe deployment expectations and should not be interpreted as completed deployment features unless they have been configured and verified.

IX. TESTING ARCHITECTURE

The project uses Pytest to automate verification of application behavior. The test organization described in the project materials is summarized in Table VI.

The project team reported a test execution on November 10, 2026, with 57 tests passed and 3 subtests passed in 28.01 seconds. These results represent the reported local test execution and should be retained only if they match the final repository test output.

GitHub Actions is used for automated workflow execution. The workflow status should be checked in the repository before final submission.

TABLE VII
ARCHITECTURE AND REQUIREMENTS TRACEABILITY

SRS Requirement	Supporting Component
FR-01 to FR-03	Authentication and session management
FR-04	Dashboard component
FR-05 to FR-08	Inventory management component
FR-09 to FR-12	Inventory display and filtering
FR-13	Stock classifier
FR-14	Availability reporter
FR-15	CSV exporter
FR-16 to FR-17	Validation and database constraints
NFR-01	Presentation layer and user interface
NFR-02	Application request processing
NFR-03	SQLite persistence
NFR-04	Layered architecture and modular organization
SEC-01 to SEC-06	Authentication, validation, session and deployment security

X. DEPLOYMENT ARCHITECTURE

The application runs locally using the Python Flask development server, on the configured local port. The development server supports local development and testing but is not intended for production deployment.

A production deployment may place the WSGI application behind a production server such as Gunicorn and a reverse proxy such as Nginx, with SSL/TLS configured. These components describe an intended deployment approach and should not be interpreted as already deployed unless the team has configured them.

The database file must be stored in an appropriate persistent location and included in a backup strategy.

XI. REQUIREMENTS TRACEABILITY

The architecture supports the major functional and non-functional requirements identified in the SRS. Table VII provides a high-level mapping between requirements and architectural components.

XII. QUALITY ATTRIBUTES

The architecture addresses the following quality attributes:

- **Maintainability:** Functional responsibilities are separated across templates, backend logic, and database access.
- **Reliability:** SQLite persistence and database constraints support consistent inventory records.
- **Security:** Password hashing, session-based access control, input validation, and parameterized queries provide the documented security baseline.
- **Testability:** Pytest suites and GitHub Actions support repeatable automated verification.
- **Usability:** Dashboard summaries, filters, stock classifications, reports, and CSV export support inventory tasks.
- **Portability:** Python and SQLite provide a platform-independent application foundation, subject to dependency compatibility.

XIII. Risks and Limitations

SQLite is suitable for a lightweight project deployment, but concurrent write workloads may require additional database planning. The local Flask development server is not intended as a production server.

Production use requires deployment hardening, HTTPS configuration, secure secret management, backups, and operational monitoring. Performance targets and cross-platform behavior require measurement in the intended deployment environment.

The architecture document describes logical components and intended responsibilities. The final implementation should be checked against the source code to ensure that all described behavior is present.

XIV. Conclusion

The SEIMS architecture provides a three-layer web application that separates presentation, application processing, and data persistence. Flask handles routing and business logic, while SQLite stores user and component information. Authentication, inventory management, stock classification, reporting, CSV export, and automated testing form the main architectural capabilities.

The design provides a maintainable foundation for the project's stated requirements. Production deployment, security hardening, and performance validation remain subject to implementation and verification.

XV. References

References

[1] IEEE, *IEEE Standard for Information Technology—Systems Design—Software Design Descriptions*, IEEE Std 1016-2009, 2009.

[2] SEIMS Project Team, "Software Requirements Specification for the Electronics Store Inventory Management System," PES University, 2026.

[3] Pallets Projects, "Flask Documentation." [Online]. Available: https://flask.palletsprojects.com/

[4] SQLite Development Team, "SQLite Documentation." [Online]. Available: https://www.sqlite.org/docs.html

[5] Pytest Development Team, "Pytest Documentation." [Online]. Available: https://docs.pytest.org/

[6] Python Software Foundation, "Python Documentation." [Online]. Available: https://docs.python.org/